Reproducing GlareShell: Graph Learning-based PHP Webshell Detection, Revisited under Leakage Control

NT230 – Malware analysis course project

Đặng Minh Tú – 24520033 Võ Quang Vũ – 24522045 Lê Trí Đức – 24520009
Instructor: Phan Thế Duy

October 2026

[TODO: abstract – after GNN runs]

1 Introduction

Webshells are the standard persistence mechanism after a web-application compromise. GlareShell [6] proposes to detect PHP webshells statically by turning each script into an interprocedural control-flow graph (ICFG) of PHP statements, attributing each node with a word embedding of the normalised statement and a risk level, and classifying the graph with a GNN. It reports an F1 of 0.9887 with GAT, above sequence- and tree-based approaches.

This report reproduces GlareShell from its description – neither code nor data were released – and then asks whether the reported performance measures what it claims to measure. Our contributions:

1. **A working re-implementation** on top of the same static-analysis tool (LChecker), including four extractor problems that silently corrupt the graphs if left unfixed (Section 3).
2. **A rebuilt, documented corpus** (GlareShell-R) from the paper's sources with provenance, two deduplication levels, near-duplicate clusters, a data card and a label guide (Section 4).
3. **A faithful replication** of the paper's Tables 2 and 4 and Figures 6–8 with five seeds and confidence intervals (**??**).
4. **A leakage and generalisation analysis**: a cluster-disjoint split, a cross-source test on four unseen benign projects, simple baselines, and an error taxonomy (**????**).

[TODO: summary of findings – after GNN runs]

2 The malware object: PHP webshells

2.1 What a webshell is and how it operates

A *webshell* is a server-side script planted in the document root of a web server that gives a remote party interactive control of that server through ordinary HTTP requests (MITRE ATT&CK T1505.003 [11]). It is a *post-exploitation persistence* artefact: it does not compromise the server by itself but is dropped after an initial-access step (unrestricted file upload, LFI/RFI, SQL injection with `INTO OUTFILE`, a vulnerable plugin, stolen FTP/CMS credentials). Once present, the PHP interpreter executes it on every request to its URL, with the privileges of the web-server user. GlareShell [6] places this in an Industrial-Internet setting: the web server is the management and monitoring interface of industrial control systems, so a webshell is a pivot from the IT web tier towards the OT network.

Figure 1 summarises the concepts we use throughout the report. Webshells fall into a few *families* that differ in size and in the evidence they leave in source code:

- **One-line backdoors** ("China Chopper"-style): `<?php @eval($_POST['x']);?>`. One statement; the attacker's client sends the PHP payload in every request.
- **Command shells**: pass request input to `system`/`exec`/`passthru`/`popen`/`` `backticks` ``.
- **Management kits** (c99, r57, WSO, b374k): thousands of lines with file manager, upload, SQL client, port binding, mail bomber, self-removal. Most of their code is ordinary UI code.
- **Obfuscated loaders**: hide the payload in an encoded string (`eval(gzinflate(base64_decode('...')))`) or build the dangerous function name at run time (`$f='as'.'sert'; $f($_REQUEST[k]);`), or route it through callbacks (`array_map($_GET['f'], [$_GET['a']])`, `preg_replace('/.*/e', ...)`).
- **Uploaders / file managers**: no command execution at all – only `move_uploaded_file`, `fopen`/`fwrite`; they are used to stage the next payload.

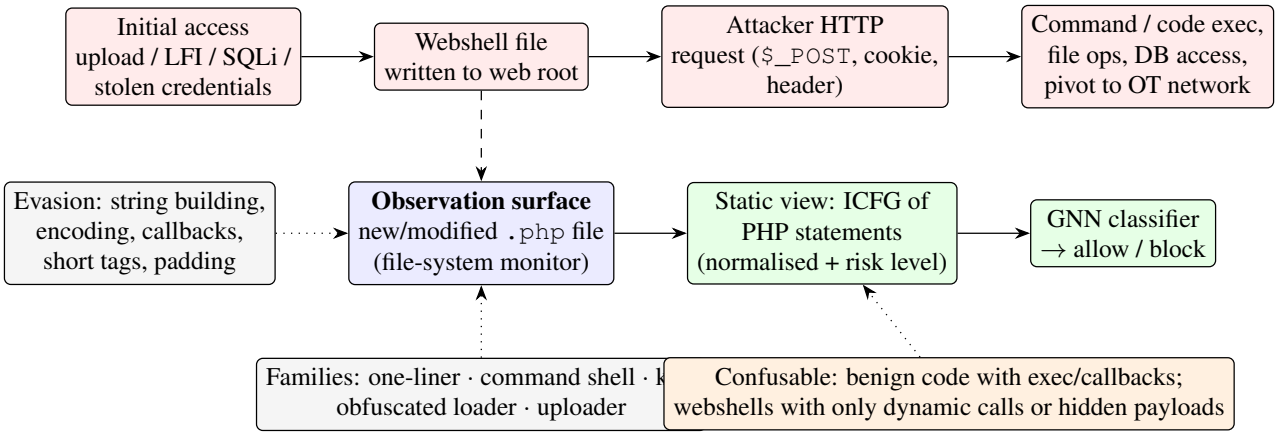


Figure 1: Concept map: attack chain (red), what the detector observes (blue), the detector (green) and the sources of confusion we analyse (orange).

Variants inside a family usually differ only by password, banner, language of the UI or layout, which is why public collections are full of near-duplicates (Section 4).

2.2 Threat model

- **Assets**: the web server of an Industrial-Internet management plane and the networks it can reach.
- **Attacker**: remote, already able to write a file into the web root; knows that a static detector may be deployed and can obfuscate, but (in the paper's setting) does not adapt to the specific model.
- **Defender's observation**: the PHP source file at creation/modification time. No execution traces, no HTTP logs, no file-system context (path, owner, neighbouring files).
- **Decision**: binary, per file, before the file is executed; a false positive blocks a legitimate deployment, a false negative leaves a backdoor.
- **Out of scope**: memory-resident shells, non-PHP shells, adaptive adversarial attacks against the model, legitimately deployed admin tools.

2.3 Research questions

RQ1 (reproduction) Can GlareShell's reported detection performance (Tables 2 and 4 of [6]) be reproduced from the paper's description, using the paper's data sources and protocol?
RQ2 (components) Do statement normalisation, risk weights, the choice of embedding and the choice of GNN contribute as claimed (Fig. 6–8 of [6])?
RQ3 (validity) How much of the performance is explained by near-duplicate leakage between train and test and by the benign/webshell source confound?
RQ4 (generalisation) How does the detector behave on benign projects and webshell collections that were not part of the training sources, i.e. the false-positive rate a deployment would see?

2.4 Labels and the features the detector sees

A file is *webshell* if it comes from a public webshell collection and *benign* if it comes from a pinned release of a widely deployed project; labels are source-level (full guide in `docs/LABEL_GUIDE.md`). Table 1 lists what GlareShell actually observes for each file.

Confusable cases. The feature view predicts where the detector should fail, and we test these predictions in **??**: (i) benign framework code is full of *obfuscated*-level calls (`call_user_func_array` for dispatch, `usort` with closures, `preg_replace_callback`) and installers/updaters call `exec`; (ii) webshells that never call a dangerous built-in by name (dynamic calls `$f(...)`, uploaders) get no risk signal; (iii) payloads inside encoded strings are a single opaque statement; (iv) the static ICFG is incomplete for reflection, variable functions and `include` of runtime paths.

Table 1: Feature view of GlareShell (per node of the ICFG unless stated).

Feature	Content	Malicious signal it can carry
Statement text	One PHP statement / branch condition, pretty-printed	API used, request input, payload strings
Normalisation	local variables → `$local`, user functions/methods → `user_define`, numbers → `im`	removes names/passwords, keeps built-ins and strings
Embedding	100-d vector of the normalised statement (W2V / FastText / GloVe / D2V)	co-occurrence of risky statements
Risk level	one-hot: 2 = *critical* (`system`, `exec`, `shell_exec`, `passthru`, `popen`, `proc_open`, `pcntl_exec`, backticks); 1 = *obfuscated* (`eval`, `assert`, `preg_replace`, `create_function`, `call_user_func*`, array/sort callbacks, ...); 0 = normal	explicit dangerous capability
Edges	intra-procedural control flow + call/return edges between functions (ICFG)	how input reaches the sink
Graph	mean of final node states (Eq. 6 of [6])	whole-file behaviour

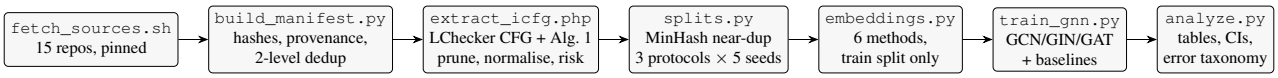


Figure 2: Reproduction pipeline. Samples are only parsed; nothing is executed.

3 GlareShell and our re-implementation

3.1 The original pipeline

GlareShell [6] has three components. **(1) ICFG extractor**: LChecker [9] (built on PHP-Parser [13]) builds one CFG per function/method; Algorithm 1 of the paper connects them through call relations into an interprocedural CFG whose nodes are PHP statements; "meaningless" statements (constant `echo`, variable initialisation) are pruned. **(2) Node attribution generator**: statements are normalised (user functions → `user-define`, local variables → `local`, numbers → `im`) and embedded with one of six word-embedding methods (Word2Vec CBOW/Skip-gram [10], FastText [3], GloVe [12], Doc2Vec PV-DM/PV-DBOW [8]); a three-level risk weight is concatenated. **(3) GNN classifier**: GCN [7], GAT [14] or GIN [15], mean pooling and an MLP. The paper reports FastText as best embedding (F1 0.9454 with a small GCN) and GAT as best GNN (F1 0.9887) on 3,104 webshells / 10,375 benign files split randomly 8:1:1. Neither code nor data is published.

3.2 Our implementation

Figure 2 shows the reproduction pipeline; every box is a script in `src/`.

ICFG extraction. We tried LChecker first, as the paper did. Its *analysis* (`Main.php`) is a taint analysis for loose comparisons and does not output graphs, but its CFG builder (`ConstructCFG.php`) is reusable as a library. We therefore call LChecker's `ConstructCFG`, `ConstructFunctionCFG` and `ConstructClassCFG` unchanged and implement the rest of the paper on top (`src/php/extract_icfg.php`, 400 lines):

1. **Algorithm 1**: lower the main-script CFG into a node/edge list; for every statement, resolve calls to functions, methods (by name, $\leq$3 candidates), constructors and closures defined in the same file, lower the callee CFG once and add a call edge (site → callee entry) and a return edge (callee exit → site). Dynamic calls (`$f()`) cannot be resolved statically – as in the paper.
2. **Pruning**: constant `echo`/inline HTML and assignments of constants to variables are removed; their predecessors are linked to their successors so the graph stays connected.
3. **Normalisation**: applied on a copy of the AST (not on text), so string literals keep their content; a function is "user-defined" if it is not a built-in of PHP 8.3 or of a common extension (prefix list).
4. **Risk level**: computed on the original AST from the called built-ins (Table 1).

LChecker is a research prototype written for PHP 7 and PHP-Parser 4.3. Applying it to this corpus revealed four problems that the paper does not mention and that we had to fix to obtain meaningful graphs:

- it *silently skips* `namespace`, `try/catch` and `declare` blocks and functions declared inside other statements. Every PSR-style framework file is wrapped in a namespace, so vanilla LChecker yields empty graphs for most of Laravel/Drupal – a perfect label leak. We flatten these blocks before CFG construction;
- a `for` loop's condition is stored as a PHP array, which crashes pretty-printing (854 files); we keep its last expression;

Table 2: Corpus funnel per source. "exact" = first copy per SHA-256 (the paper's dedup); "ws" = first copy per whitespace-insensitive hash. Duplicates are resolved in favour of the paper sources, so the small numbers for the individual webshell collections mean that their files are already inside Cyc1e183.

Label	Subset	Source	PHP-ext files	contain <?	exact dedup	ws dedup
benign	heldout	joomla/joomla-cms	3105	3105	3104	3103
benign	heldout	laravel/framework	2193	2100	2100	2100
benign	heldout	matomo-org/matomo	3085	3071	2946	2946
benign	heldout	phpmyadmin/phpmyadmin	951	951	951	951
benign	paper	PHPMailer/PHPMailer	124	124	123	123
benign	paper	WordPress/WordPress	1452	1452	1448	1448
benign	paper	doorgets/CMS	1573	1568	1449	1443
benign	paper	drupal/drupal	9868	9866	9860	9860
benign	paper	laravel/laravel	51	50	50	50
webshell	heldout	BlackArch/webshells	201	186	15	1
webshell	heldout	xl7dev/WebShell	282	278	31	18
webshell	paper	Cyc1e183/PHP-Webshell-Dataset	2917	2908	2908	1645
webshell	paper	JohnTroony/php-webshells	119	110	27	3
webshell	paper	bartblaze/PHP-backdoors	184	181	33	10
webshell	paper	tennc/webshell	381	378	129	89

- PHP 8.3 ships with `short_open_tag=Off`, so files written with bare `<?` tags parse as HTML. This affected 208 webshells and *zero* benign files; we parse with `short_open_tag=On`;
- 2 files crash the PHP interpreter (segfault on deeply nested expressions); they are kept as empty graphs.

We cap an ICFG at 2,000 statements to bound memory (Table 3).

Embeddings and GNNs. Gensim 4.3.3 provides Word2Vec, FastText and Doc2Vec with default settings (dimension 100, window 5, 5 epochs, `min_count`=1), except FastText's hash-bucket count (200k instead of 2M, for RAM) and PV-DBOW's `dbow_words`=1 (otherwise the statement vectors are never trained). GloVe is our own PyTorch implementation (x_{max}=100, α=0.75, 25 AdaGrad epochs) because `glove-python` no longer installs. Each file's statements in emission order form one "sentence". **Embeddings are trained only on the training files of each split and seed** (the paper does not say; training them on all files would leak test statements). Unknown statements get a zero vector, except for FastText which composes them from character n-grams. GNNs are implemented with PyTorch Geometric 2.6.1 following Eq. (3)–(6) of the paper; edges are made undirected; the node attribute is [embedding $\|$ one-hot risk]. Table 4 (appendix) lists all hyper-parameters; they follow the paper where it gives values (Table 2 setting: 2-layer GCN, hidden 10, batch 10; Table 3 optimum: 3 layers, lr 10^{-3}, batch 256, 120 epochs, hidden 32/8×8 heads/16). Model selection uses the validation F1 (early stopping); the paper does not describe its selection.

4 Dataset: rebuilding the corpus and measuring its quality

We take the *Dataset Research Track*: besides rebuilding the paper's corpus we release a provenance-documented manifest with two deduplication levels, near-duplicate clusters and two additional evaluation protocols ("GlareShell-R"; data card in `docs/DATA_CARD.md`, schema in `docs/SCHEMA.md`, label guide in `docs/LABEL_GUIDE.md`).

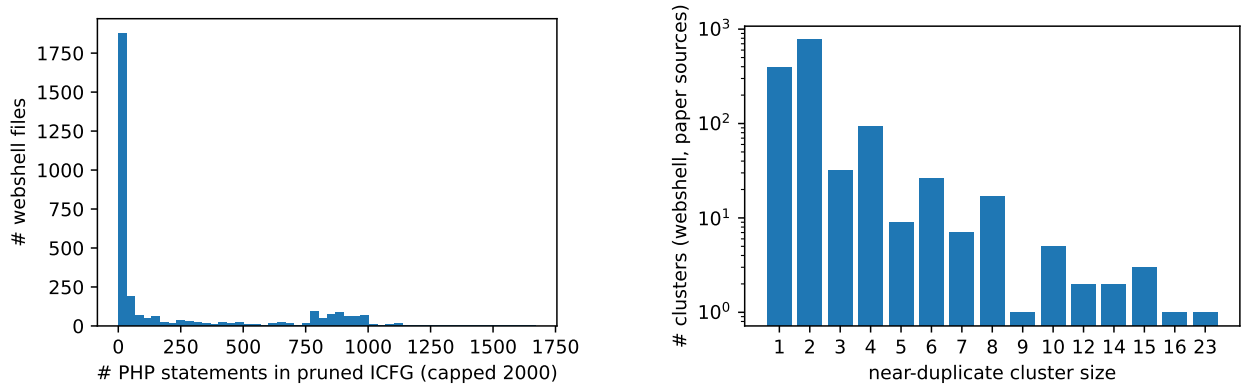
4.1 Sources and provenance

We cloned every repository of the paper's Table 1 at pinned revisions (Table 2). `ysrc/webshell-sample` has been deleted from GitHub; its files survive only inside the Cyc1e183 aggregate [5], a "cleaned" set of 2,917 webshells drawn from 17 collections, which dominates the webshell side. Its files are named by MD5 and carry no origin; matching MD5s against the collections we could clone recovers the origin of only 646 of them (2,271 remain *unknown*). Benign files come from pinned releases (WordPress 6.4.3, Drupal 10.2.5, Laravel skeleton v10.3.3, PHPMailer 6.9.1, doorGets CMS). For the dataset track we add four *held-out* benign projects (Laravel framework v10.48.4, phpMyAdmin 5.2.1, Joomla 5.0.3, Matomo 5.0.3) and two held-out webshell collections (BlackArch, xl7dev). Licenses range from CC0/MIT to GPL; several webshell collections state none, which is one more reason we never redistribute samples publicly (Section 7).

Reproduced composition. After exact-hash deduplication the paper sources yield **3,097 webshells** (paper: 3,104) and 12,930 benign files. The paper reports 10,375 benign files without saying how they were chosen; we subsample proportionally per project (seed 2024) to exactly that size. The webshell count matches the paper within 0.2%, which suggests the paper used *exact-hash* deduplication.

Table 3: ICFG extraction outcome (after the four extractor fixes of Section 3).

Label	Subset	Files	Crash	Parse err.	Empty ICFG	Truncated	Median nodes	P90 nodes	ms/file
benign	heldout	9101	0	1	123	7	18	103	6.9
benign	paper	12930	0	1	590	49	14	99	7.9
webshell	heldout	46	0	7	3	4	25	755	26.6
webshell	paper	3097	4	139	53	477	11	875	36.6



(a) Statements per webshell ICFG (cf. paper Fig. 5).

(b) Near-duplicate cluster sizes, paper webshells.

Figure 3: Corpus structure.

4.2 ICFG extraction statistics

Table 3 shows that extraction itself is *label-dependent*: webshells are older, less well-formed and much larger. 139 webshells (4.5%) contain syntax errors that PHP-Parser recovers from, 477 (15.4%) exceed the 2,000-statement cap (benign: 0.4%), and 53 produce an empty graph. These are signals a classifier can exploit without understanding behaviour (graph size), and failure modes that a real deployment must handle (an unparsable upload is itself suspicious). Statement counts of webshells (Figure 3a) span four orders of magnitude, as in the paper's Fig. 5. Extraction takes 11 ms per file on average (P95 40 ms) on one core.

4.3 Near-duplicates and leakage

Exact hashing is a weak notion of "unique" for scripts that are copied, re-saved with different line endings, re-branded or re-passworded. We measure duplication at two further levels:

- **Whitespace-insensitive hash**: 1,265 of the 2,917 Cyc1e183 files (43%) are byte-identical once whitespace is removed – the "cleaned" dataset was deduplicated on raw bytes only.
- **Near-duplicate clusters**: we represent each file by the *set* of its normalised ICFG statements, generate candidates with MinHash-LSH [4] (128 permutations, 32 bands) and merge pairs whose exact Jaccard similarity is ≥ 0.8 (union-find). The 3,097 paper webshells collapse into **1,368 clusters**; only 394 webshells are singletons and the largest family has 55 members (Figure 3b). Benign files are much less redundant (12,930 files $\rightarrow$ 11,828 clusters). No cluster mixes labels.

Under the paper's protocol (random 8:1:1), **77.0% $\pm$ 4.4% of test webshells have a near-duplicate in the training set** (5 seeds; benign: 9.0% $\pm$ 1.4%). The paper's test set therefore mostly measures recognition of already-seen families [1, 2].

4.4 Evaluation protocols

`paper_random` the paper's protocol: exact dedup, 3,097 webshells / 10,376 benign, stratified random 8:1:1.
`clean_group` leakage-controlled: whitespace dedup (1,747 webshells / 10,372 benign); whole near-duplicate clusters are assigned to train, validation or test (8:1:1 by file count, per label). Same sources as the paper.
`cross_source` generalisation: train/validation of `clean_group`; test = held-out projects/collections after removing anything that shares a cluster with the paper sources: 9,091 benign files from four unseen projects and **only 18** unseen webshells. The latter is too small to estimate recall precisely (we give Wilson intervals); `cross_source` is primarily a *false-positive* stress test, which is the quantity that decides deployability.
Each protocol is instantiated with 5 seeds. We could not build a meaningful temporal split: git "first-seen" dates are upload dates, not malware dates – all 2,917 Cyc1e183 files share one commit (2020-11-13, a single zip), 98% of xl7dev and 86% of BlackArch files share one day; only `tennc/webshell` has a spread history (100 distinct days, 2013–2023), and its files are mostly already inside the aggregate. Source-disjointness (`cross_source`) is our substitute.

5 Experimental setup

Hardware/software. Intel CPU (16 threads), 7.6 GB RAM, NVIDIA RTX 3050 Laptop 6 GB, WSL2 Ubuntu, PHP 8.3.6, Python 3.11, PyTorch 2.4.1 (CUDA 12.1), PyTorch Geometric 2.6.1, Gensim 4.3.3 (paper: Xeon E5-2620 for extraction, i7-12700 + RTX 3060 for training).

Metrics. Positive class = webshell. We report the paper's Precision, Recall, TNR, Accuracy, F1 (Eq. 7–11) plus the false-positive rate FPR = 1 − TNR, which matters most in deployment because benign uploads vastly outnumber webshells. Every configuration is run with **5 seeds** (split, embedding and GNN initialisation all depend on the seed; sweeps use 3 seeds); we report mean ± half-width of the 95% Student-t interval. For `cross_source` we additionally give 95% Wilson intervals on the pooled test counts of seed 0.

Experiments.
1. **Table 2 replication**: 6 embeddings × 5 seeds, 2-layer GCN, hidden 10, batch 10 (`paper_random`).
2. **Fig. 8 replication**: ablation *none* (raw statements, no risk), *n* (normalised), *n*+*w* (normalised + risk), FastText + GCN (`paper_random`) and FastText + GAT (`clean_group`).
3. **Fig. 6/7 replication**: embedding dimension 20–120 and context window 2–7 (FastText + GCN, 3 seeds).
4. **Table 4 replication**: GCN/GIN/GAT with the Table 3 optimum, FastText (`paper_random`).
5. **Beyond the paper**: GCN and GAT on `clean_group` and `cross_source`; baselines on all protocols.

Baselines (`src/baselines.py`, same splits): **TF-IDF + LR / XGBoost** on lexical PHP tokens of the raw file (1–2-grams, hashed to 2^{18} features; χ^2 top-5k for XGBoost) – the classic text approach of prior work [6]; **rules** derived from GlareShell's own risk mechanism (webshell iff a critical call / any risky call / a risky call and request input); and **graph-statistics + XGBoost**: 12 numbers per ICFG (size, density, #critical, #obfuscated, unique-statement ratio, parse error, truncated, . . .) and *no code semantics*. If this probe performs well, part of the task is solvable from artefacts of the corpus rather than from behaviour. [TODO: results – after GNN runs]

6 Discussion

6.1 What was reproduced, improved, or not reproduced

[TODO: verdict table: reproduced / improved / not reproduced – after GNN runs]

6.2 Threats to validity

Internal validity (does our pipeline do what the paper did?).
- *Unreleased implementation.* ICFG construction above LChecker's CFG builder, the pruning rules, the built-in function list, the risk lists and the statement text format are our reading of the paper. Different choices change the vocabulary (we obtain 484k distinct normalised statements) and therefore the embeddings.
- *Extractor fixes.* We changed LChecker's behaviour on namespaces, try/catch, `for` conditions and short tags (Section 3). Each fix was needed to avoid graphs that are empty for one class only; if the authors ran vanilla LChecker on PHP 8 their graphs were partly empty, and their numbers are not directly comparable to ours in either direction.
- *Unspecified training details* (epochs for Table 2, early stopping, model selection, embedding training corpus, threshold). We fixed them before looking at test results and report all of them (Table 4).
- *Resource-driven deviations*: 2,000-statement cap (15% of webshells truncated), FastText 200k buckets, own GloVe implementation.

Construct validity (do the labels and metrics measure webshell detection?).
- *Source-level labels.* "Benign" means "file of a large, modern framework/CMS release", "webshell" means "file of a public collection". The two populations also differ in age (PHP 4/5 vs. PHP 8 style), size, structure (procedural vs. namespaced classes) and parseability (Table 3). A classifier can reach high scores by separating these *populations* rather than by recognising malicious behaviour; the graph-statistics probe and the hand-written standalone benign scripts (**??**) quantify this.
- *Label noise.* Collections contain non-malicious helper files; benign projects contain test fixtures and installers with webshell-like capabilities. We did not relabel by hand.
- *Class prior.* 23% webshells in test is unrealistically high for a file-upload monitor, so precision and F1 overstate deployment performance; FPR on the 9,091 held-out benign files is the more relevant number.

External validity (does it generalise?).
- Only five benign projects in training; four more in testing. Industrial web servers (the paper's motivating setting) run vendor-specific, often small, procedural PHP applications that resemble neither population.
- Only 18 webshells are novel w.r.t. the paper sources; recall on unseen families is therefore estimated with very wide intervals. No temporal split was possible (Section 4.3).
- No adaptive attacker: renaming variables and functions is neutralised by normalisation, but string building, dynamic calls and payload encoding already defeat the risk mechanism for many collected samples.

Conclusion validity (are the differences real?). Five seeds give 95% intervals of roughly ± 0.01–0.03 F1 for the GNNs; several differences reported in the paper (e.g. between embedding methods, 0.91–0.95 F1 in a single run) are of the same order and cannot be distinguished without repeated runs. We never select hyper-parameters on the test set.

7 Reproducibility, safety and ethics

Handling live malware. The corpus contains working webshells. Our rules (also in `README.md`): samples are *only parsed* by PHP-Parser or read as bytes – no script includes, evaluates or serves them, and the extractor is invoked on lists of paths, never inside a web server; `samples/` and every derived file that contains code fragments (`data/processed/`) are git-ignored; the Docker image runs with `--network none` and the samples mounted read-only; for hand-over to graders, `scripts/pack_samples.sh` builds a password-protected archive (password `infected`, the customary convention) with a SHA-256 checksum, to be shared privately. The development VM can be rolled back from a snapshot after use. The repository publishes only hashes, paths, aggregate statistics and code. We do not release any new webshell, payload or evasion technique; the evasion classes we discuss are already documented in the public collections.

Licenses. Benign projects are GPL/LGPL/MIT and are re-fetched from upstream rather than redistributed. Several webshell collections state no license; we reference them by URL and commit only.

Reproducibility artefacts. `scripts/fetch_sources.sh` (pinned revisions), `scripts/setup_tools.sh` (LChecker @c8a3646, PHP-Parser v4.19.4, no composer needed), `requirements.txt` (exact versions), `Dockerfile`, `scripts/reproduce.sh` (end-to-end), the manifest with SHA-256 of every sample (integrity check), the splits as JSON, every run's configuration, metrics and per-file predictions as JSON (`results/runs/`), and `src/analyze.py`, which regenerates every table and figure of this report. Total runtime on our laptop: [TODO: hours]. [TODO: conclusion – after GNN runs]

References

[1] Miltiadis Allamanis. The adverse effects of code duplication in machine learning models of code. In *Onward! (SPLASH)*, 2019.

[2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don'ts of machine learning in computer security. In *USENIX Security Symposium*, 2022.

[3] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *TACL*, 5:135–146, 2017.

[4] Andrei Z. Broder. On the resemblance and containment of documents. In *Compression and Complexity of Sequences*, 1997.

[5] Cyc1e183. PHP-Webshell-Dataset. https://github.com/Cyc1e183/PHP-Webshell-Dataset, commit 59ff14f, 2021.

[6] Pengbin Feng, Dawei Wei, Qiaoyang Li, Qin Wang, Youbing Hu, Ning Xi, and Jianfeng Ma. GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet. *Computer Networks*, 245:110406, 2024.

[7] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.

[8] Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *ICML*, 2014.

[9] Penghui Li and Wei Meng. LChecker: Detecting loose comparison bugs in PHP. In *Proceedings of The Web Conference (WWW)*, 2021.

[10] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *NeurIPS*, 2013.

[11] MITRE ATT&CK. Server software component: Web shell (T1505.003). https://attack.mitre.org/techniques/T1505/003/, 2024.

[12] Jeffrey Pennington, Richard Socher, and Christopher Manning. GloVe: Global vectors for word representation. In *EMNLP*, 2014.

[13] Nikita Popov. PHP-Parser: A PHP parser written in PHP. https://github.com/nikic/PHP-Parser, v4.19.4, 2024.

[14] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *ICLR*, 2018.

[15] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *ICLR*, 2019.

A Hyper-parameters

Table 4: Hyper-parameters. "Paper" = value stated in [6]; everything else is our choice.

Item	Value	Source
Split	8:1:1 stratified (`paper_random`), cluster-grouped (`clean_group`)	paper / ours
Embedding dim / window	100 / 5 (sweeps 20–120 / 2–7)	paper (default, Fig. 6–7)
Embedding epochs	gensim default (5; Doc2Vec 10), GloVe 25	default
Table 2 GNN	GCN, 2 layers, hidden 10, batch 10, lr 10^{-3}	paper (lr ours)
Table 2 epochs	$\leq$50, early stop on val. F1 (patience 10)	ours
Table 3/4 GNN	3 layers, lr 10^{-3}, batch 256, $\leq$120 epochs (patience 30)	paper (patience ours)
Hidden	GCN 32, GIN 16, GAT 8×8 heads	paper
Optimiser / loss	Adam / cross-entropy, threshold 0.5	ours
Pooling	mean (Eq. 6), then MLP $d \to d \to 2$	paper / ours
ICFG cap	2,000 statements	ours
Near-dup threshold	Jaccard ≥ 0.8 on normalised statement sets	ours
Seeds	0–4 (sweeps 0–2)	ours

B Mapping to the course rubric

Criterion	Where
1. Malware object & RQ	Section 2: concept map (Figure 1), families, threat model, RQ1–4, labels, feature table, confusable cases
2. Reproduction	Section 3, **??**: code in `src/`, environment, comparison tables with the paper, ablation, sweeps, 5 seeds
3. Dataset	Section 4, `docs/DATA_CARD.md`, `LABEL_GUIDE.md`, `SCHEMA.md`, manifest, cluster file, splits, fetch/clean scripts
4. Experiments & critique	**??** and section 6: P/R/F1/FPR with CIs, confusion matrices, error taxonomy, cross-source evaluation, threats to validity
5. Reproducibility & ethics	Section 7, `README.md`, `Dockerfile`, checksums, `pack_samples.sh`